

Spherepop: A Language for Geometric Computation

Unified Specification

Flyxion

July 26, 2026

Abstract

Spherepop is a formal calculus and computational architecture in which transformation is represented through explicit structural operations rather than hidden mutation. Its central objects are bounded semantic regions, relations among those regions, and operations that introduce, transform, identify, compose, or refuse structure.

This specification develops Spherepop at several related levels. It defines a human-facing notation for expressing geometric constructions; a typed Spherepop Calculus (SPC) in which the primitive operations receive formal semantics; and an operational interpretation in which computations may be represented as explicit, replayable transformations. Derived representations, implementations, and user-facing utilities are distinguished from the authoritative semantic structure they inspect or propose to modify.

The aim is not merely to replace textual syntax with spatial notation. Spherepop treats geometry, causality, equivalence, and transformation as aspects of the computational substrate itself. This makes the history of a computation available for inspection while permitting higher-level geometric and semantic interpretations to be reconstructed from the same underlying structure.

Contents

1	Overview	2
2	The Primitive Operators	3
3	Geometric DSL (Front-End)	5
3.1	Grammar (EBNF)	5
3.2	Semantics-by-surface	6
4	The Derived Surface Calculus	6
4.1	Syntax	7
4.2	Typing Rules	7
4.3	Collapse Rules and the Quotient Construction	7
4.4	Desugaring into the Four Primitives	8
4.5	Functional Completeness	9
4.6	Operational Semantics of the Surface Calculus	9

5	Type-Theoretic Layer	9
6	From Calculus to Event Algebra	10
6.1	Authoritative History, Replay, Proposals, Observation	10
6.2	Utility Taxonomy	11
7	Reference Implementation (Haskell)	11
7.1	Core AST	11
7.2	Typing (Sketch)	11
8	Kernel Reference Implementation (Rust)	12
8.1	Event Types and ABI Layout	12
8.2	History, Option Space, and Replay	13
8.3	Collapse Rules	14
8.4	Arbiter and Proposals	15
8.5	Overlay Manager and Preview-Commit	16
8.6	Worked Trace	17
9	Geometric Semantics	17
10	Pipeline	18
11	Example	18
11.1	DSL	18
11.2	Lowered Core	18
11.3	Explanation	18
A	Appendix A: Typing Derivations	18
B	Appendix B: Operational Semantics Proofs	19
C	Appendix C: Category-Theoretic Interpretation	19

1 Overview

Spherepop begins from the observation that conventional programming languages encode much of their structure indirectly. Scope, adjacency, dependency, identity, evaluation order, and transformation history are represented through combinations of textual position, naming conventions, mutable state, and implicit runtime mechanisms. Spherepop instead investigates how much of this structure can be made explicit within the computational representation itself.

The basic intuition is geometric, but “geometry” is used here in a structural rather than merely graphical sense. A sphere denotes a bounded region of admissible structure. A pop denotes an explicit transformation or passage across such a boundary. Merge expresses an identification or equivalence operation whose precise admissibility must be specified by the calculus. Relations may connect otherwise distinct objects without thereby identifying them. More elaborate operations are constructed from, or constrained by, these primitive structural transformations.

This distinction is important because Spheredpop is not simply a visual syntax for an underlying conventional language. The surface notation is replaceable. The semantic operations are not. A conforming implementation may therefore expose Spheredpop through textual, graphical, interactive, or programmatic interfaces provided that these interfaces preserve the same underlying calculus.

The architecture consequently distinguishes three questions that an implementation must not conflate. The first is what transformation has been proposed or performed. The second is what semantic structure follows from that transformation. The third is how that structure is presented to an observer. A rendering, summary, query result, serialization, or visualization may be recomputed or discarded without changing the structure from which it was derived.

Where Spheredpop is implemented through an authoritative event history, this distinction becomes operational. Semantic changes are represented explicitly as events or proposals rather than hidden mutations. Replaying the same admissible history must reconstruct the same semantic state. Speculative transformations may be represented separately from authoritative history and inspected before commitment. These implementation constraints provide one concrete realization of the more general Spheredpop principle that transformation history should remain distinguishable from derived representation.

The remainder of this specification develops these layers separately. It first defines the authoring notation, then gives the syntax and typing discipline of the core calculus, then specifies its operational behavior. Subsequent sections relate the calculus to replayable computation, geometric interpretation, implementation, and higher-level utilities. Mathematical interpretations are introduced only where the corresponding algebraic properties have been explicitly established.

2 The Primitive Operators

This section is the foundation on which every later layer of this specification is built, and it supersedes the primitive vocabulary of earlier Spheredpop documents. **Pop, Refuse, Bind, and Collapse are the only primitive operators.** Sphere, Merge, Choice, Link, Unlink, Nest, and SetMeta all of which appear as primitives somewhere in the prior Spheredpop literature are second-order syntactic sugar, each expressible as a composition of these four. This is not a stylistic preference; it is a completeness claim, argued below in the same spirit as functional completeness for Boolean connectives or Church-style encodings in the untyped λ -calculus: a small basis is shown sufficient, and everything built on a larger, historically-accreted vocabulary is shown reducible to it.

Definition 1 (Spheredpop World). A *Spheredpop world* is a pair $W = (H, \Omega)$ where $H \in \mathcal{H}$ is a history a finite sequence of events over a fixed alphabet E and $\Omega \subseteq \Omega_0$ is the current option space, drawn from a finite initial option space Ω_0 .

Definition 2 (Event Alphabet). $E = \{\text{Pop}\} \cup \{\text{Refuse}\} \cup \{\text{Bind}\} \cup \{\text{Collapse}\}$.

Definition 3 (The Four Operators).

$$\begin{aligned}
P_x &: (H, \Omega) \mapsto (H ++ [\text{Pop}(x)], \Omega \setminus \{x\}) & (\text{Pop: commitment}) \\
R_{x,r} &: (H, \Omega) \mapsto (H ++ [\text{Refuse}(x, r)], \Omega) & (\text{Refuse: documented inadmissibility}) \\
B_{a,b} &: (H, \Omega) \mapsto (H ++ [\text{Bind}(a, b)], \Omega) & (\text{Bind: dependency declaration}) \\
C_{x,c} &: (H, \Omega) \mapsto (H ++ [\text{Collapse}(x, c)], \Omega) & (\text{Collapse: observation under rule } c)
\end{aligned}$$

P_x is defined only for $x \in \Omega$ and forecloses every future in which x takes a different value. $R_{x,r}$ leaves Ω untouched a refusal documents that a path was considered and rejected, for reason r , without foreclosing structural availability; a refusal without a reason documents nothing. $B_{a,b}$ records that two elements are coupled without consuming either in particular $\text{Bind}(a, b)$ never identifies a and b : *relatedness is not identity*. $C_{x,c}$ requires an admissibility certificate, $\Gamma \vdash t : \text{Adm}((\cdot)T)$; a term whose type is not $\text{Adm}((\cdot))$ cannot be collapsed, and this is a type error rather than a runtime failure.

Remark 1 (Collapse is not the roadmap’s COLLAPSE). An earlier kernel-facing document (the December 2025 utility roadmap) uses **COLLAPSE** to name *bulk equivalence* a batched identification of objects, essentially Merge at scale. That is a different operation from the primitive **Collapse** defined here, which is *observation under a rule*: a functor from histories to a quotient space $O_c = \mathcal{H}/\sim_c$ (§4.3), gated by an admissibility certificate. The two senses must not be conflated, exactly as calculus-**Pop** (application, in the original 2025 specification) must not be conflated with kernel-**POP** (object introduction, in the roadmap). Where this specification needs the roadmap’s bulk-equivalence notion, it is written as *batch identification* and is derived sugar (§4.4, Def. 12), not a primitive.

Theorem 1 (Completeness of the Four Operators). $\{P_x, R_{x,r}, B_{a,b}, C_{x,c}\}$ suffice to express variable assignment, conditional branching, function application, exception handling, memory deallocation, and concurrent event recording.

Proof sketch. *Assignment* $x := v$: initial assignment is P_v followed by $B_{x,v}$ commit to the value, then bind the name to it. *Reassignment* $x := v'$ against an existing binding is $R_{B_{x,v}, \text{Reassigned}}$ followed by $P_{v'}$ then $B_{x,v'}$: the prior binding is refused (documented as superseded, not deleted Corollary 1) before the new one is committed.

Conditional on ϕ : P_{ϕ^+} if ϕ holds; otherwise $R_{\phi^+, \text{ConstraintViolation}(\phi)}$ followed by P_{ϕ^-} the refusal records *why* the positive branch was not taken. Choice (§4.4) is the special case where *both* the taken and rejected branches are always recorded: $P_A \parallel R_B$.

Function application $f(a)$: $\text{Collapse}_{c_{\text{subst}}}(B_{a,x})$ no separate lambda-specific event kind is needed (§4.4); binding the argument to the parameter and collapsing under the substitution rule both performs substitution and forces the commitment.

Exceptions: $R_{t,r}$ documented inadmissibility rather than system destruction; the reason r is a first-class certificate.

Deallocation: $R_{p, \text{Deallocated}}$ for pointer symbol p , followed by $C_{p, c_{\text{reclaim}}}$ a quotient collapse that recovers option-space bounds for future allocation without releasing the historical trace itself. Use-after-free becomes a type error rather than a runtime fault, and the deallocation remains auditable rather than erased.

Concurrency: two proposal streams over a shared Ω are unified via $B_{a,b}$, coupling their results as dependent rather than by any separate synchronization primitive. $\square$

Remark 2 (Meld is structural, not part of the concurrency proof). Independently of the derivation above, $(\mathcal{H}, ++, \varepsilon)$ together with a tensor $\otimes$ is a free strict monoidal category

on histories $H_1 \otimes H_2$ is the parallel composition of two independently-generated histories at the structural level. This is a true fact about $\mathcal{H}$'s algebra, but it is not needed to prove Theorem 1's concurrency clause, which goes through **Bind** alone.

Theorem 2 (Conservation of Possibility). Define $w : E \rightarrow \mathbb{N}$ by $w(\text{Pop}(x)) = 1$, $w(\text{Refuse}) = w(\text{Bind}) = w(\text{Collapse}) = 0$, and $\Pi(H, \Omega) = |\Omega| + \sum_{e \in H} w(e)$. For any legal execution sequence, $\Pi(H_t, \Omega_t) = |\Omega_0|$ for all t : possibility is conserved, never created or destroyed only consumed by Pop or documented as inadmissible by Refuse. Bind and Collapse affect neither $|\Omega|$ nor w .

Corollary 1 (Irreversibility). $|H_{t+1}| > |H_t|$ for every step, since every operator appends exactly one event. No legal execution can return to a prior world state.

Remark 3. This is the formal correlate of the history-first ontology stated in the Overview: a world that could return to a prior state would be one in which the intervening history was undone, which is not computation but its negation.

3 Geometric DSL (Front-End)

3.1 Grammar (EBNF)

```

program      ::= { scene | comment } ;
scene        ::= "@scene" "{" { stmt } "}" ;

stmt         ::= sphere_decl | link_decl | unlink_decl | refuse_decl
               | spin_decl | burst_decl | pop_decl | choose_decl
               | let_decl | comment ;

sphere_decl  ::= "sphere" IDENT "(" { attr ("," attr)* } ")" ;
attr         ::= IDENT ":" value ;
let_decl     ::= "let" IDENT "=" expr ;

link_decl    ::= "link" IDENT op IDENT [ "[" IDENT "]" ] ;
op           ::= "->" | "" | "" | "" | "" ;

unlink_decl  ::= "unlink" IDENT op IDENT ;
refuse_decl  ::= "refuse" expr [ "because" STRING ] ;

spin_decl    ::= "spin" IDENT "(" { attr ("," attr)* } ")" ;
burst_decl   ::= "burst" IDENT "(" { arg ("," arg)* } ")" ;
pop_decl     ::= "pop" IDENT [ "with" IDENT ] [ "when" condition ] ;
choose_decl  ::= "choose" NUMBER ":" expr "|" expr ;

condition    ::= expr ;
expr         ::= term { ("+"|" -") term } ;
term         ::= factor { ("*"|" /") factor } ;
factor       ::= IDENT | NUMBER | STRING | "(" expr ")" ;

arg          ::= value ;

```

```

value          ::= NUMBER | STRING | IDENT | vector | tuple ;
vector         ::= "(" NUMBER "," NUMBER "," NUMBER ")" ;
tuple          ::= "(" { value ("," value)* } ")" ;

comment        ::= "#" { ANY_CHAR except newline } ;
IDENT          ::= (letter | "_") { letter | digit | "_" } ;
NUMBER         ::= digit { digit | "." digit } ;
STRING         ::= ''' { ANY_CHAR except ''' } ''' ;
letter         ::= "A".."Z" | "a".."z" ;
digit          ::= "0".."9" ;

```

3.2 Semantics-by-surface

Construct	Compilation to SPC
<code>sphere f(type: x:A.B, body: T)</code>	<code>Sphere($x:A.T$)</code>
<code>sphere c(type: A, value: v)</code>	<code>$c := v : A$</code>
<code>link a -> b</code>	<code>Link(a, b)</code> (typed relation, no identification)
<code>link a b</code>	<code>Merge(a, b)</code>
<code>link a b</code>	<code>$(a, b) : \Sigma x : A. B(x)$</code> (pairing mode)
<code>link a b</code>	<code>Pop($\nabla, \langle a, b \rangle$)</code>
<code>link a b</code>	macro: composition of two Pop-chains
<code>unlink a -> b</code>	<code>Unlink(a, b)</code>
<code>refuse e because "..."</code>	<code>Refuse(e)</code>
<code>pop f with u</code>	<code>Pop(f, u)</code>
<code>burst g(a,b,...)</code>	nested Pop
<code>choose p: t u </code>	proposal-level combinator over t, u (see §6)
<code>spin x(,limit)</code>	macro: <code>fix F</code> or <code>iterate n F(x)</code>
<code>@scene {...}</code>	program root Γ of declarations/terms

Explanation. The DSL is only notation. All meaning is inherited from the SPC core. The overloaded `link` keyword lowers to different primitives depending on its operator: identifies (`Merge`), pairs (-introduction), applies over a pair (`Pop`), composes without introducing a new primitive, and the previously unassigned `->` now denotes `Link`, a typed relation that connects two objects *without* identifying them. `Unlink` is the structural dual of `Link`, and `Refuse` records that a proposed transformation was rejected as inadmissible rather than silently discarded. Merging requires type equality; `Link` requires only that the relation's type is declared, not that its endpoints share a type. Choice is deferred to the kernel section: at the DSL/SPC level it names a proposal over two candidate continuations, not a term that itself reduces to a value.

4 The Derived Surface Calculus

Everything in this section is sugar. None of the operators below `Sphere`, application-`Pop` (in its *term-calculus* sense, as opposed to the primitive commitment operator of §2), `Merge`, `Choice`, `Link`, `Unlink`, `SetMeta`, `Nest` is primitive. Each is shown below to reduce to a composition of `Pop`, `Refuse`, `Bind`, `Collapse`. Retaining this layer's typed term syntax is useful precisely because it gives programmers the familiar shapes (abstraction, application, identification, probabilistic branching) without requiring every program to

be written directly against the possibility-space primitives the same relationship NAND-complete Boolean circuits bear to AND/OR/NOT, or Church encodings bear to the untyped λ -calculus: a convenient vocabulary built entirely from a minimal basis.

4.1 Syntax

Definition 4 (Terms).

$$t, u ::= x \mid a \mid \text{Sphere}(x:A.t) \mid \text{Pop}(t, u) \mid \text{Merge}(t, u) \mid \text{Choice}(p, t, u) \\ \mid \text{Link}(t, u) \mid \text{Unlink}(t, u) \mid \text{Refuse}(t).$$

4.2 Typing Rules

Definition 5 (Typing).

$$\begin{array}{ll} \text{Var} & \frac{(x:A) \in \Gamma}{\Gamma \vdash x : A} \quad \text{Atom} \frac{}{\Gamma \vdash a : A} \\ \Pi\text{-intro} & \frac{\Gamma, x:A \vdash t : B}{\Gamma \vdash \text{Sphere}(x:A.t) : \Pi x:A. B} \quad \Pi\text{-elim} \frac{\Gamma \vdash f : \Pi x:A. B \quad \Gamma \vdash u : A}{\Gamma \vdash \text{Pop}(f, u) : B[u/x]} \\ \text{Merge} & \frac{\Gamma \vdash t : A \quad \Gamma \vdash u : A}{\Gamma \vdash \text{Merge}(t, u) : A} \quad \text{Choice} \frac{\Gamma \vdash t : A \quad \Gamma \vdash u : A}{\Gamma \vdash \text{Choice}(p, t, u) : A} \\ \Sigma\text{-intro} & \frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B(t)}{\Gamma \vdash \langle t, u \rangle : \Sigma x:A. B(x)} \\ \text{Link} & \frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B}{\Gamma \vdash \text{Link}(t, u) : \text{Rel}(A, B)} \quad \text{Unlink} \frac{\Gamma \vdash \text{Link}(t, u) : \text{Rel}(A, B)}{\Gamma \vdash \text{Unlink}(t, u) : \mathbf{1}} \\ \text{Refuse} & \frac{\Gamma \vdash t : A}{\Gamma \vdash \text{Refuse}(t) : \text{Refused}(A)} \end{array}$$

These rules govern the *surface* type system that programmers see. They are consistent, but they are not foundational: §4.4 gives each construct's reduction to **Pop/Refuse/Bind/Collapse**, and the primitive-level guarantees (Theorem 1, Theorem 2) are what actually back these surface types, not the other way around.

4.3 Collapse Rules and the Quotient Construction

The Merge- and SetMeta-sugar below both depend on the primitive **Collapse**'s parameterization by a *collapse rule*.

Definition 6 (Collapse Rule and Observational Equivalence). A collapse rule is a function $c : \mathcal{H} \rightarrow \mathcal{O}_c$ from histories to an observational space. Histories H_1, H_2 are observationally equivalent under c , written $H_1 \sim_c H_2$, iff $c(H_1) = c(H_2)$.

Definition 7 (Observable State as Quotient). The observable state space under c is $\mathcal{O}_c = \mathcal{H}/\sim_c$: an observable state is an equivalence class of histories, not a history itself.

Remark 4. Different rules make different distinctions relevant. The identity rule $c_I(H) = H$ makes every event visible; coarser rules discard information deliberately. Merge, defined below, is what happens when the chosen rule identifies two bound elements.

4.4 Desugaring into the Four Primitives

Definition 8 (Abstraction and Application). $\text{Sphere}(x : A. t) := \text{Bind}(x, \Omega)$: abstraction freezes/scopes the option space relative to x by binding x to it, rather than by any separate lambda-specific event. Application is

$$\text{Pop}(f, u) := \text{Collapse}_{c_{\text{subst}}}(\text{Bind}(u, x)),$$

where c_{subst} is the substitution rule identifying the bound variable x with the argument u : the bind couples argument to parameter, and the collapse under c_{subst} both performs the substitution and forces the commitment P_u . There is no separate E_λ fragment: abstraction and application compile to **Bind** and **Collapse** alone, with no third event kind smuggled in for functions.

Definition 9 (Choice). $\text{Choice}(A, B) := \text{Pop}(A) \parallel \text{Refuse}(B)$: choosing A over B is the parallel composition of committing to A *and* explicitly, auditably refusing B . Both events land in H . This is stronger than treating the untaken branch as simply absent from history: Choice is a special case of conditional branching (Theorem 1) in which the rejected alternative is always documented, never silently dropped.

Definition 10 (Link). $\text{Link}(t, u) := \text{Bind}(t, u)$, exactly. Link is not a distinct operator: it is the name this surface calculus gives to **Bind** when the DSL author is thinking in terms of "relations" rather than "dependencies." The $\text{Rel}(A, B)$ surface type is bookkeeping, not evidence of a different underlying primitive.

Definition 11 (Unlink). $\text{Unlink}(t, u) := \text{Refuse}(\text{Bind}(t, u))$. Withdrawing a relation is documented inadmissibility of continued reliance on a prior bind, following exactly the pattern Theorem 1 already uses for deallocation ($R_{p, \text{Deallocated}}$). This does not remove the original **Bind** event from H consistent with Irreversibility (Corollary 1) it only adds a refusal that later collapse rules may honor by excluding withdrawn bindings from their observable state.

Remark 5 (Refuse graduates). $\text{Refuse}(t)$ as written in the surface grammar above is not sugar for anything further it *is* the primitive **Refuse**, wrapped only in enough surface typing ($\text{Refused}(A)$) to keep it inside the term calculus's bookkeeping.

Definition 12 (Merge). $\text{Merge}_c(a, b) := \text{Collapse}_c(\text{Bind}(a, b))$: Merge is the projection of a **Bind** under an identification-quotient rule c . Idempotence ($\text{Merge}_c(t, t) \equiv t$) and commutativity follow from c 's equivalence-relation structure rather than being posited as separate axioms. Batch identification (the roadmap's **COLLAPSE**) is the same construction generalized to n bound elements under one c , confirming that sense was never a primitive.

Definition 13 (SetMeta, proposed). $\text{SetMeta}(o, k, v) := B_{o, (k, v)}$ under a distinguished collapse rule c_{meta} that every other collapse rule is defined to ignore by convention: ordinary observation is blind to metadata bindings unless a rule explicitly opts in. This is the one construct in this section still marked proposed rather than confirmed against an external derivation.

Definition 14 (Nest). **Nest** is sugar for a chain of $\text{Bind}/\text{Collapse}_{c_{\text{subst}}}$ pairs nested abstraction/application matching its treatment as "nested Pop" in the DSL table above.

4.5 Functional Completeness

Theorem 3 (NOR is representable). Represent Boolean values as commitments to canonical option handles $\omega_T, \omega_F \in \Omega$. Then $\text{NOR}(A, B)$ is representable in $\mathcal{P}$.

Proof. If neither A nor B committed ω_T , then $\text{Refuse}(A)$ and $\text{Refuse}(B)$ both succeed without conflict and $\text{Pop}(\omega_T)$ executes: output True. If either committed ω_T , the corresponding refusal guard detects that ω_T was already committed and is itself inadmissible, so execution falls back to $\text{Pop}(\omega_F)$: output False. Since $\{\text{NOR}\}$ is functionally complete for Boolean logic, all Boolean connectives are representable in $\mathcal{P}$. $\square$

Theorem 4 (β -reduction equivalence). For terms M, A : evaluating $\text{Pop}(\text{Sphere}(x:A. M), A)$ under the desugaring above yields the same result as $M[x \mapsto A]$.

Proof. By the Abstraction/Application definition, $\text{Pop}(\text{Sphere}(x.M), A) = \text{Collapse}_{c_{\text{subst}}}(\text{Bind}(A, x))$. The bind couples A to x ; c_{subst} 's quotient identifies x with A throughout M 's bound option space and the resulting collapse performs exactly the substitution $M[x \mapsto A]$, discharging the commitment in the same step. $\square$

4.6 Operational Semantics of the Surface Calculus

Definition 15 (Reduction).

- () $\text{Pop}(\text{Sphere}(x:A. t), u) \longrightarrow t[u/x]$
- (Idem) $\text{Merge}(t, t) \longrightarrow t$ (inherited from $c_{\sim}$, see Def. 12)
- (Prob) $\text{Choice}(p, t, u)$ resolves to t w.p. p , u w.p. $1 - p$.

Remark 6. There is no separate (Cancel) axiom for Unlink : since $\text{Unlink}(t, u)$ is just $\text{Refuse}(\text{Bind}(t, u), \cdot)$, and Refuse -terms are normal forms at the primitive level (§2), unlinking simply does not reduce further—it sits in H as a durable, queryable record, exactly like any other refusal.

Explanation. Every reduction rule here is a restatement, at the surface level, of something already guaranteed by the primitive layer: β -reduction is deferred commitment discharging; Merge's idempotence is quotienting under $c_{\sim}$; Choice's branching is proposal-time resolution that never touches H until a branch is taken.

5 Type-Theoretic Layer

Judgements and Meta-Theory. We assume a standard cumulative hierarchy of universes, admissible substitution, and the usual structural rules. Subject reduction (preservation) and progress hold for the deterministic fragment; the stochastic fragment preserves types in expectation.

Theorem 5 (Preservation). If $\Gamma \vdash t : A$ and $t \longrightarrow t'$, then $\Gamma \vdash t' : A$.

Theorem 6 (Progress). If $\Gamma \vdash t : A$ then t is a value or there exists t' with $t \longrightarrow t'$.

Explanation. Well-typed programs do not “go wrong”: reduction does not change types, and every non-value advances. **Choice** introduces stochastic steps but not type ambiguity.

6 From Calculus to Event Algebra

Where §2 gives the possibility-space semantics of **Pop**, **Refuse**, **Bind**, **Collapse** and §4 shows every other operator reducing to them, this section states what an implementation must actually persist and replay. Because the surface calculus is now *fully* desugared into the primitive alphabet, this section is considerably simpler than earlier drafts of this specification: there is no longer a separate kernel vocabulary to reconcile against a calculus vocabulary. Kernel-**Pop** and calculus-**Pop** are the same operator, by construction, because the primitive layer was built first and the surface layer was derived from it rather than the reverse.

6.1 Authoritative History, Replay, Proposals, Observation

Definition 16 (Authoritative history). An authoritative history is a finite sequence $H \in \mathcal{H}$ of events drawn from the fixed alphabet $E = \{\text{Pop}, \text{Refuse}, \text{Bind}, \text{Collapse}\}$ (§2), totally ordered by sequence position.

Definition 17 (Replay function). $R : \text{Event}^* \rightarrow (\text{State} \times \wp(\Omega_0))$ maps a history to the world it denotes, by folding the four operators exactly as defined in §2: $R(\varepsilon) = (s_0, \Omega_0)$ and $R(H \frown e) = \delta(R(H), e)$.

Definition 18 (Proposals). A proposal is an event sequence $p \in \text{Event}^*$ not yet appended to H .

Definition 19 (Observation as Collapse invocation). Observing H under rule c is invoking the primitive $C_{x,c}$ (§4.3); there is no separate, non-authoritative “view” operator. Invoking Collapse both (a) appends a **Collapse** event to H , recording that an observation under c occurred, and (b) yields the observable value $c(H) \in O_c$.

Invariant 1 (Deterministic replay). $H_1 = H_2 \implies R(H_1) = R(H_2)$.

Invariant 2 (Total causal order). Every $e_i \in H$ has a well-defined position i ; δ may depend on $R(e_0, \dots, e_{i-1})$ but never on later positions or wall-clock time.

Requirement 1 (Authoritative discipline). No utility computes δ against H directly. Authoritative changes are submitted as proposals to an arbiter, which alone decides whether a proposal is appended.

Requirement 2 (Observation non-interference). The arbiter’s acceptance decision for any proposal must not depend on the *observable value* $c(H)$ produced by any prior Collapse only on (H, Ω) directly. The fact that a Collapse occurred, and under which rule, is an ordinary recorded event like any other; what must never happen is a later acceptance decision being a function of *what was seen*, only of what was done.

Conformance Criterion 1. An implementation conforms only if Invariant 1 is empirically falsifiable (replay H twice, compare) and Requirement 2 is enforced structurally: the arbiter’s validation function must have no parameter through which $c(H)$ for any c can be passed in, even though **Collapse** events are visible to it as history.

Remark 7 (Two collisions, both now resolved). Earlier drafts of this specification carried two unresolved terminological collisions. First, the roadmap’s kernel event **POP** (“introduce a semantic object handle”) versus calculus-**Pop** (application) resolved because application is now sugar (Def. of Abstraction and Application, §4.4) for a deferred instance

of the one true primitive **Pop**. Second, the roadmap’s **COLLAPSE** (bulk equivalence) versus the primitive **Collapse** (observation under a rule) resolved because bulk equivalence is Merge-sugar generalized to n elements (Def. 12), not a primitive event at all. Neither collision required renaming anything; both were dissolved by correctly identifying which side of each pair was primitive.

6.2 Utility Taxonomy

Definition 20 (Utility classes). Relative to H and R , a utility is exactly one of:

- a *proposal generator*: produces $p \in \text{Event}^*$ without appending it;
- an *observer*: invokes **Collapse** under some rule c and returns $c(H)$, producing no further proposal;
- an *overlay manager*: creates, rebases, or discards a proposal p held outside H , without itself deciding acceptance.

Requirement 3 (Preview-commit). An overlay manager exposing a preview-then-commit workflow must never commit by default; commitment requires a separate, explicit call that routes through the arbiter as an ordinary proposal.

7 Reference Implementation (Haskell)

7.1 Core AST

```
data Tm
  = Var Name
  | Atom Name
  | Sphere Name Ty Tm
  | Pop Tm Tm
  | Merge Tm Tm
  | Choice Double Tm Tm
  | Link Tm Tm
  | Unlink Tm Tm
  | Refuse Tm
```

7.2 Typing (Sketch)

```
infer (Sphere x A t) = Pi x A (infer (,x:A) t)
infer (Pop f u)      = case infer f of
  Pi x A B | infer u == A -> subst x u B
  _ -> error "Not a function"
infer (Merge t u)    = mustEqual (infer t) (infer u)
infer (Choice p t u) = mustEqual (infer t) (infer u)
infer (Link t u)     = Rel (infer t) (infer u)
infer (Unlink t u)   = case infer (Link t u) of
  Rel _ _ -> Unit
  _ -> error "Not a link"
infer (Refuse t)     = Refused (infer t)
```

Explanation. The implementation follows the rules verbatim. **Sphere** synthesizes a Π -type; **Pop** eliminates it. **Merge** and **Choice** check branch types for equality. **Link** produces a relation type from two independently-typed terms; **Unlink** only type-checks against a term that is itself a well-typed **Link**; **Refuse** wraps a type in **Refused** rather than discarding it. This Haskell model is deliberately scoped to SPC alone – it has no event log, no replay, and no notion of an arbiter. Those belong to the kernel, given a separate implementation in §8 because they are not properties of the term calculus but of how an authoritative history is engineered around it.

8 Kernel Reference Implementation (Rust)

Where §6 gives the authoritative-history apparatus its formal shape, this section gives it an executable one. The Haskell model above demonstrates that SPC has an intelligible reduction semantics; this Rust sketch demonstrates that the surrounding kernel – event log, replay, arbiter, overlays – can be engineered to the invariants and requirements stated above. The two implementations are not alternative encodings of the same thing: Haskell has no event log, and this Rust sketch performs no β -reduction of its own – it persists and replays events, and treats calculus-level reduction as something a proposal generator does before it ever emits an event.

8.1 Event Types and ABI Layout

Every event carries a stable discriminant, a logical position, and a payload whose layout must not change without a version bump. There are exactly four kinds now – no **Merge**, **Link**, **Unlink**, or **SetMeta** variant exists at this layer, since §4.4 shows all four to be sugar. **Pop** here is unambiguously the primitive commitment operator; the earlier homonym with a distinct “object introduction” kernel event no longer arises; abstraction and application (§4.4) both compile to it.

```
#[repr(u8)]
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum EventKind {
    Pop      = 0, // commitment: remove x from Omega
    Refuse    = 1, // documented inadmissibility of x, with reason
    Bind      = 2, // coupling of (a, b), optionally tagged with a relation label
    Collapse  = 3, // observation of H under a named collapse rule
}

pub type ObjectId = u64;
pub type LogPos   = u64;
pub type RuleId   = &'static str; // identifies a registered collapse rule c

#[derive(Clone, Debug)]
pub struct Event {
    pub kind: EventKind,
    pub pos:  LogPos,           // position in H; assigned only at commit time
    pub a:    Option<ObjectId>, // Pop's x; Bind's first element; Refuse's target
    pub b:    Option<ObjectId>, // Bind's second element
}
```

```

    pub tag:    Option<String>,    // Bind's relation label (Link-sugar uses this)
    pub reason: Option<String>,    // Refuse's rationale required, never optional i
    pub rule:   Option<RuleId>,    // Collapse's rule c
}

```

Remark 8. ABI stability means: no field above may be removed, reordered, or reinterpreted in a released version. New fields are additive; a decoder must skip unrecognized fields rather than fail closed. Because the alphabet is now fixed at four kinds by Theorem 1 rather than accreted from an evolving roadmap, this layout is expected to be far more stable across versions than the seven-variant enum an earlier draft of this specification carried.

8.2 History, Option Space, and Replay

```

#[derive(Default, Clone)]
pub struct State {
    pub option_space: std::collections::HashSet<ObjectId>, // Omega
    pub committed:   std::collections::HashSet<ObjectId>, // popped symbols
    pub bound:       std::collections::HashSet<(ObjectId, ObjectId, String)>,
    pub refused:     Vec<(LogPos, ObjectId, String)>,
    pub observed:    Vec<(LogPos, RuleId)>, // audit trail of Collapse invocations
}

pub struct History {
    events: Vec<Event>,
}

impl History {
    pub fn replay(&self, omega_0: &std::collections::HashSet<ObjectId>) -> State {
        let mut s = State { option_space: omega_0.clone(), ..State::default() };
        for e in &self.events {
            apply(&mut s, e);
        }
        s
    }
}

fn apply(s: &mut State, e: &Event) {
    match e.kind {
        EventKind::Pop => {
            let x = e.a.unwrap();
            s.option_space.remove(&x); // |Omega| decreases by exactly one
            s.committed.insert(x);
        }
        EventKind::Refuse => {
            s.refused.push((e.pos, e.a.unwrap(), e.reason.clone().unwrap_or_default()));
            // Omega is untouched: refusal documents, it does not foreclose structural
        }
        EventKind::Bind => {

```

```

        s.bound.insert((e.a.unwrap(), e.b.unwrap(), e.tag.clone().unwrap_or_default))
    }
    EventKind::Collapse => {
        // Collapse does not mutate committed/bound/refused: it only records that
        // an observation occurred. The observable value c(H) is computed separately
        // (see collapse rules below), never stored in State itself.
        s.observed.push((e.pos, e.rule.unwrap()));
    }
}
}

```

Remark 9. `apply` is a pure function of `(State, Event)` no wall-clock reads, no ambient globals which is what makes Invariant 1 checkable rather than merely asserted. Note that `Collapse`'s branch deliberately records *that* an observation happened and *under which rule*, but never the observed value itself: that keeps Requirement 2 satisfiable, since nothing in `State` the thing `apply` and validation ever see carries $c(H)$.

8.3 Collapse Rules

Collapse rules are pure functions of `History`, external to `apply`, matching §4.3. Merge-sugar (Def. 12) is realized as one such rule.

```

/// c_~ : identifies objects connected by a Bind, realizing Merge-sugar.
pub fn collapse_quotient(h: &History) -> UnionFind {
    let mut uf = UnionFind::new();
    for e in h.events_of_kind(EventKind::Bind) {
        uf.union(e.a.unwrap(), e.b.unwrap());
    }
    uf // 0_c for this rule: the partition of ObjectId into Merge-equivalence classes
}

```

```

/// c_meta : the distinguished rule other rules are defined to ignore (SetMeta-sugar).
pub fn collapse_meta(h: &History) -> std::collections::HashMap<ObjectId, Vec<(String,
    h.events_of_kind(EventKind::Bind)
    .filter(|e| e.tag.as_deref() == Some("__meta__"))
    .fold(Default::default(), |mut acc, e| {
        acc.entry(e.a.unwrap()).or_insert_with(Vec::new)
        .push((e.tag.clone().unwrap(), String::new()));
    })
    acc
}

```

```

/// c_I : the identity/finest rule every event visible, nothing quotiented.
pub fn collapse_identity(h: &History) -> &[Event] { h.as_slice() }

```

Remark 10. Only Collapse events with a registered, admissibility-certified rule may be committed (see `Arbiter::validate` below) an unregistered rule is a type error at proposal time, not a runtime panic at observation time, matching the primitive definition's precondition $\Gamma \vdash t : \text{Adm}(()T)$.

8.4 Arbiter and Proposals

```
pub struct Proposal {
  pub events: Vec<Event>,    // positions unassigned; filled in at commit
}

pub struct Arbiter {
  history: History,
  rules:   std::collections::HashSet<RuleId>, // the admissibility-certified rule
}

#[derive(Debug)]
pub enum ArbiterError {
  Malformed(String),
  PopOutsideOptionSpace,
  UncertifiedCollapseRule,
  StaleOverlay,
}

impl Arbiter {
  pub fn state(&self, omega_0: &std::collections::HashSet<ObjectId>) -> State {
    self.history.replay(omega_0)
  }

  /// Validates and, if accepted, appends a proposal to H. The ONLY path by
  /// which H is ever extended.
  pub fn submit(&mut self, p: Proposal, omega_0: &std::collections::HashSet<ObjectId>)
    -> Result<Vec<LogPos>, ArbiterError>
  {
    self.validate(&p.events, omega_0)?;
    let mut positions = Vec::new();
    for mut e in p.events {
      let pos = self.history.len() as LogPos;
      e.pos = pos;
      self.history.push(e);
      positions.push(pos);
    }
    Ok(positions)
  }

  fn validate(&self, events: &[Event], omega_0: &std::collections::HashSet<ObjectId>)
    -> Result<(), ArbiterError>
  {
    let s = self.history.replay(omega_0); // structural state only never c(H) f
    for e in events {
      match e.kind {
        EventKind::Pop if !s.option_space.contains(&e.a.unwrap()) =>
          return Err(ArbiterError::PopOutsideOptionSpace),
      }
    }
  }
}
```

```

        EventKind::Collapse if !self.rules.contains(&e.rule.unwrap()) =>
            return Err(ArbiterError::UncertifiedCollapseRule),
        _ => {}
    }
}
Ok(())
}
}

```

Remark 11. `validate` reads `state()`, which carries only (H, Ω) -level facts never the value any collapse rule would compute. There is no parameter through which $c(H)$ could be passed in, for any c : this is Requirement 2 enforced by the type signature, not by convention.

8.5 Overlay Manager and Preview-Commit

```

pub struct Overlay {
    base_len: usize,          // H.len() at the time the overlay was created
    pending: Proposal,
}

pub struct OverlayManager<'a> {
    arbiter: &'a mut Arbiter,
}

impl<'a> OverlayManager<'a> {
    pub fn create(&self, pending: Proposal) -> Overlay {
        Overlay { base_len: self.arbiter.len(), pending }
    }

    /// Non-authoritative: replays H + overlay without touching H.
    pub fn preview(&self, o: &Overlay, omega_0: &std::collections::HashSet<ObjectId>) {
        let mut speculative = self.arbiter.history_clone();
        for e in o.pending.events.clone() {
            speculative.push(e);
        }
        speculative.replay(omega_0)
    }

    /// The only call that can make an overlay authoritative routes through
    /// Arbiter::submit like any other proposal.
    pub fn commit(&mut self, o: Overlay, omega_0: &std::collections::HashSet<ObjectId>)
        -> Result<Vec<LogPos>, ArbiterError>
    {
        if o.base_len != self.arbiter.len() {
            return Err(ArbiterError::StaleOverlay); // H moved since preview
        }
        self.arbiter.submit(o.pending, omega_0)
    }
}

```



```
}
```

Remark 12. There is deliberately no `auto_commit` method and no default argument making `commit` implicit in `create` or `preview`: “no utility may auto-commit by default” is the absence of a code path here, not a documented convention.

8.6 Worked Trace

```
let omega_0: HashSet<ObjectId> = [1, 2].into_iter().collect();
let mut arb = Arbiter::new(["merge_quotient", "identity"]);

arb.submit(Proposal { events: vec![pop(1), pop(2)] }, &omega_0)?; // commit a, b
arb.submit(Proposal { events: vec![bind(1, 2, "adjacent")] }, &omega_0)?; // Bind(a,b)

// Observe under the quotient rule: this IS Merge, per Def. of Merge-sugar.
arb.submit(Proposal { events: vec![collapse("merge_quotient")] }, &omega_0)?;
let classes = collapse_quotient(arb.history_ref());
assert_eq!(classes.find(1), classes.find(2)); // a and b are now one class

// Withdraw reliance on the bind without deleting it (Unlink-sugar):
arb.submit(Proposal {
  events: vec![refuse_bind(1, 2, "adjacent", "relation withdrawn")]
}, &omega_0)?;
assert!(arb.state(&omega_0).bound.contains(&(1, 2, "adjacent".into()))); // H unchanged
assert!(arb.state(&omega_0).refused.iter().any(|(_,_,r)| r == "relation withdrawn"));
```

Explanation. The quotient-collapse call demonstrates that Merge never needed to be a fifth event kind: identifying a and b is what a specific, admissibility-certified observation rule does with an ordinary **Bind** event already in H . The final refusal demonstrates Unlink-sugar: the original bind is untouched (Irreversibility, Corollary 1), and a later collapse rule is free to honor the refusal by excluding withdrawn bindings from its own observable state a policy decision for that rule, not a structural deletion.

9 Geometric Semantics

Reading the Primitives as Geometry. The primary geometric reading now belongs to §2, not to the surface calculus: **Pop** is the contraction of the option space Ω narrowing possibility; **Refuse** marks a region of Ω as excluded without contracting it; **Bind** draws an edge between two points without merging them; **Collapse** is a projection of the whole history onto an observational plane O_c , with different rules c giving different projections of the same underlying object. The surface calculus’s older reading **Sphere** as a local region, **Merge** as entropic smoothing, **Choice** as bifurcation still holds, but now derivatively: it describes what the desugared primitive events look like when composed in the specific patterns §4.4 gives for abstraction, quotienting, and proposal-time branching.

10 Pipeline

Stage	Input	Output
Parse	DSL Scene	AST (Scene)
Desugar	AST	SPC Terms
Typecheck	SPC Terms	Typed Terms in Γ
Evaluate	Typed Terms	Normal Forms (+ stochastic)
Interpret	Terms	Field configuration $(\Phi, \mathbf{v}, \mathcal{S})$

Explanation. Authoring, compilation, and execution are strictly layered; correctness resides at the SPC level, independent of surface notation.

11 Example

11.1 DSL

```
@scene {
  sphere f(type: x:A.B, body: pop g with x)
  sphere g(type: x:A.B, value: <primitive>)
  sphere a(type: A, value: a0)

  pop f with a
  choose 0.5: pop g with a | pop f with a
}
```

11.2 Lowered Core

```
f = Sphere(x:A. Pop(g, x))
g = <primitive> : x:A.B
a = a0 : A
Pop(f, a)
Choice(0.5, Pop(g, a), Pop(f, a))
```

11.3 Explanation

Application collapses the abstraction boundary; choice branches probabilistically. Typing ensures both branches agree, and evaluation proceeds by -reduction.

Remark 13. The appendices below concern the derived surface calculus of §4, not the primitive layer of §2. Where they speak of “Merge” or “Choice” as though typing them directly, read this as shorthand for typing their desugared primitive forms.

A Appendix A: Typing Derivations

Application.

$$\begin{aligned} \Gamma \vdash f : \Pi x : A. B \quad \Gamma \vdash a : A \\ \text{Hence } \Gamma \vdash \text{Pop}(f, a) : B[a/x]. \end{aligned}$$

Merge.

$$\Gamma \vdash t : A \wedge \Gamma \vdash u : A \Rightarrow \Gamma \vdash \text{Merge}(t, u) : A.$$

Choice.

$$\Gamma \vdash t : A \wedge \Gamma \vdash u : A \Rightarrow \Gamma \vdash \text{Choice}(p, t, u) : A.$$

B Appendix B: Operational Semantics Proofs

Preservation (Sketch). By induction on typing derivations. The $\rightarrow$ -case follows from substitution; Merge and Choice are congruent and keep types equal.

Progress (Sketch). Values are atoms or Sphere-forms. $\text{Pop}(\text{Sphere}(\cdot), \cdot)$ reduces; otherwise subterms reduce. Stochastic steps in Choice are admissible and type-stable.

Confluence (Deterministic Fragment). Without Choice, $\rightarrow$ -reduction is confluent up to standard $\rightarrow$ -calculus arguments adapted to Merge.

C Appendix C: Category-Theoretic Interpretation

Objects and Morphisms. Types are objects; terms $t : A \rightarrow B$ are morphisms produced by Sphere; application Pop is composition up to $\equiv$.

Monoidal Structure. Merge is a symmetric, idempotent tensor: $(t\text{Merge}u)\text{Merge}v \equiv t\text{Merge}(u\text{Merge}v)$, $t\text{Merge}u \equiv u\text{Merge}t$, $t\text{Merge}t \equiv t$.

Probabilistic Functor. Choice aligns with a Girly-style monad: unit returns a degenerate choice, multiplication marginalizes nested choices.

Topos View. Interpreting Sphere-scopes as objects of a site $\mathcal{S}$, SPC terms act as arrows in the presheaf topos $\mathbf{Set}^{\mathcal{S}^{\text{op}}}$, explaining why abstraction/application behave functorially.